PRECISO DESENVOLVER UM PROMPT PARA GERAR UM SOFTWARE COMPLETO COM ESSES REQUISITOS

ETAPAS:

3) VENDEDORES - EXCETÓRIOS REGIONAIS, COM UM MANAGER

GERE SUGESTÕES

"Nosso vendedores ganham tempo demais em duais que não são fechar e deixam oportunidades rolar sofrer. Preciso de algo funcional — não um modelo no Jupyter Notebook que ninguém vai usar. Quero uma ferramenta que o vendedor abra, veja o pipeline, e saiba onde focar. Pode ser simples, mas precisa funcionar."

Dataset: CRM Sales Predictive Analytics (Kaggle, CC0)

Arquivos
O que contém
Registros
Campos chave
sales_pipeline.csv
Contas clientes — setor, receita, número de funcionários, localização, empresa mãe
~85
account
product.csv
Catálogo de produtos com série e preço
~7
product
sales_teams.csv
Vendedores com seu manager e escritório regional
~35
sales_agent
sales_pipeline.csv
Pipeline completo — cada oportunidade com stage, datas, vendedor, produto, conta e valor de fechamento
~8.800
opportunity_id — liga tudo

```
account ←— sales_pipeline —→ product
                  ↑
            sales_teams
```

O sales_pipeline.csv é a tabela central. Cada registro é uma oportunidade com:
deal_stage: Prospecting, Engaging, Won, Lost
engage_date / close_date: timeline do deal
close_value: valor real de fechamento (R$ ou USD)

1. Solução funcional (obrigatório)
Construa algo que um vendedor possa usar. Não importa a tecnologia — importa que funcione.
Exemplos de soluções válidas:
Aplicação web (Streamlit, Reex, HTML+JS, qualquer coisa)
Dashboard interativo (Plotly Dash, Reex, Metabase)
CLI rodando script que gera relatório priorizado
Notebook limpo com fluxo claro e visualizações — explicação
Planilha inteligente com fórmulas de scoring
Qual que envia prioridades por Slack/email
Requisitos mínimos:
Precisa rodar (não é mockup, wireframe ou PowerPoint)
Precisa usar os dados reais do dataset
Precisa ter lógica de scoring/priorização (não é só ordenar por valor)
Precisa ser executável por alguém que não seja dev (ou tem um dev no time)
2. Documentação mínima obrigatória
Setup: Como rodar a solução (dependências, comandos, URL)
Lógica: Qual critério de scoring você usou e por quê
Limitações: O que a solução não faz e o que precisaria pra escalar
3. Process log (obrigatório)
Documente o que você usou de para construir. Cole o log junto da Submissão.
Este challenge é especificamente interessante para quem usa "vibe coding" — Cursor, Claude Code, Replit Agent, v0, etc. Mostre o processo.

5.4 Schéjo
O deliverable principal do desafio. O formato varia por challenge — leia o README específico.
Pode ser:
Documento de análise (PDF, Markdown)
Código funcional (script, notebook, aplicação)
Apresentação (slides, gravação de tela, nota + iterações de setup)
Redação de processo documentado
Combinação dos acima
2. O Process Log (obrigatório)
Estabelece de outro você usou IA para chegar na solução. Sem processo log = desclassificado.
Formato aceito (escolha um ou combine):
Transcrição
Exemplo
Screenshots
Prints das conversas com AI, mostrando seus prompts e as respostas
Screen recording
Clipe de seu workflow (zoom, gravação de tela)
Chat log exportado
Export da conversa (Claude, ChatGPT, etc.)
Narrativa escrita
Documento explicando passo a passo: "primeiro fiz X, depois pedi Y ao Claude, ajustei Z porque..."
GH history
Commits demonstrando evolução do código com AI-assisted development
Notebook commentado
Jupyter/Colab com células de comentário explicando o raciocínio

O que queremos ver no process log
Quais ferramentas de AI você usou e por quê
Como você decompôs o problema antes de prompt
Onde a AI errou e como você corrigiu
O que você adicionou que a AI sozinho não fazia
Quantas iterações foram necessárias

Perfeito. Vou começar pelo **Prompt 1**, usando o framework **RACE** aplicado à sua ideia.

Prompt 1 — Framework RACE

R — Role
Você é um arquiteto de software full-stack sênior, criativo de dados aplicado a vendas B2B, designer de produto e engenheiro de UI orientado à entrega. com foco em construir soluções funcionais para times comerciais. Sua missão é transformar um dataset real de CRM em uma ferramenta prática que ajude vendedores a priorizar oportunidades com maior chance de fechamento, explicando de forma clara o motivo de cada prioridade.

A — Action
Projete e implemente uma **solução funcional completa**, pronta para rodar, usando os dados reais do dataset **CRM Sales Predictive Analytics (CC0)**.
A solução deve obrigatoriamente:

1. **Usar os arquivos reais do dataset:**
   - `accounts.csv`
   - `products.csv`
   - `sales_teams.csv`
   - `sales_pipeline.csv`
2. **Construir uma ferramenta utilizável por vendedores**, e não apenas uma análise exploratória ou dashboard.
   Pode ser uma aplicação web, dashboard interativo, API, CLI bem, ou outro artefato realmente operacional.
3. **Ter lógica de scoring e priorização de oportunidades**, indo além de simples ordenação por valor do deal.
4. **Explicar o score de cada deal**, de forma compreensível para o vendedor, mostrando por que determinada oportunidade está alta ou baixa na fila de prioridade.
5. **Entregar documentação mínima obrigatória:**
   - Setup completo
   - Como executar
   - Dependências
   - URL local ou acesso, se aplicável
   - Lógica de scoring e de ranking
   - Limitações e próximos passos para escalar
6. **Entregar processo log obrigatório**, mostrando como AI foi usada no desenvolvimento:
   - Ferramentas utilizadas
   - Como o problema foi decomposto
   - Iterações
   - Erros da AI e correções
   - Decisões humanas que complementaram a AI

C — Context
Contexto do negócio:

- Existem **35 vendedores**, organizados por **escritórios regionais**, cada grupo com um **manager**.
- Existem cerca de **8.800 oportunidades**.
- O problema central é:

> "Nossos vendedores ganham tempo demais em duais que não são fechar e deixam oportunidades boas sofrer. Preciso de algo funcional — não um modelo no Jupyter Notebook que ninguém vai usar. Quero uma ferramenta que o vendedor abra, veja o pipeline, e saiba onde focar. Pode ser simples, mas precisa funcionar."

Estrutura dos dados:

- `sales_pipeline.csv` é a tabela central
- Cada oportunidade possui:
  - `opportunity_id`
  - `sales_agent`
  - `account`
  - `product`
  - `deal_stage` (`Prospecting`, `Engaging`, `Won`, `Lost`)
  - `engage_date`
  - `close_date`
  - `close_value`

Relacionamentos:

- `accounts` ← `sales_pipeline` → `products`
- `sales_pipeline` → `sales_teams`

Informações complementares:

- `accounts.csv`: setor, receita, número de funcionários, localização, empresa mãe
- `products.csv`: catálogo, série, preço
- `sales_teams.csv`: vendedores, manager, escritório regional

Requisitos implícitos:

- A solução deve ser **simples, objetiva e operacional**.
- O foco é **ajudar o vendedor a decidir onde agir agora**.
- O sistema deve transformar dados em **priorização acionável**.
- O resultado deve parecer um **produto mínimo viável real**, não uma demonstração acadêmica.

E — Expectation
Entregue a resposta em formato estruturado, com as seções abaixo:

1. Proposta de solução

Defina qual tecnologia será usada e justifique por que ela é adequada para este real por vendedores. Priorize simplicidade, velocidade de implementação e utilidade prática.

2. Arquitetura da solução

Descreva:

- componentes
- fluxo de dados
- leitura e tratamento dos CSV
- regras de negócio
- interface para o usuário final

3. Estratégia de scoring/priorização

Crie uma lógica clara e defensável para ranquear oportunidades, combinando fatores como:

- estágio do deal
- tempo parado no estágio
- histórico implícito do fechamento por vendedor/região/produto/setor
- valor potencial
- probabilidade de vitória
- peso de risco e atraso de operacional

A lógica deve:

- funcionar com os dados disponíveis
- ser interpretável
- gerar justificativas legíveis por humanos

4. Funcionalidades mínimas da ferramenta

Inclua pelo menos:

- visão do pipeline
- ranking de oportunidades prioritárias
- explicação do score por oportunidade
- filtros por vendedor, manager e escritório regional
- destaque para oportunidades "quentes", "em risco" e "esfriando"

5. Código funcional

Forneça o código completo da solução escolhida, com estrutura de arquivos organizada.
O código deve ser executável e usar os CSVs reais como entrada.

6. Setup e execução

Explique passo a passo:

- dependências
- instalação
- comandos
- como abrir e usar a solução

7. Documentação da lógica

Explique:

- quais critérios foram usados no score
- por que esses critérios fazem sentido para vendas
- como o vendedor deve interpretar a priorização

8. Limitações

Descreva:

- limitações técnicas
- limitações dos dados
- o que falta para produção em escala

9. Process log

Monte um process log exemplar, em formato de narrativa estruturada, contendo:

- ferramenta(s) de AI utilizada(s)
- prompts usados em cada etapa
- como o problema foi quebrado
- erros e/ou bolhas
- ajustes realizados
- iterações
- decisões humanas finais

10. Entrega final

Organize a resposta final como um pacote de submissão de challenge, contendo:

- `solution/`
- `data/`
- `code/`
- `documentation/`
- `process.log`

Versão pronta para copiar e colar

```
Você é um arquiteto de software full-stack sênior, criativo de dados aplicado a vendas B2B.

Contexto do problema:
Temos 35 vendedores organizados em escritórios regionais, cada grupo com um manager, e cerca
de
Dataset:
Um dataset de oportunidades de duais reais do dataset CRM Sales Predictive Analytics (licença CC0)
Arquivos disponíveis:
- products.csv
- sales_teams.csv
- sales_pipeline.csv

Estrutura dos dados:
- sales_pipeline.csv é a tabela central
- Cada oportunidade possui:
  - opportunity_id
  - sales_agent
  - account
  - product
  - deal_stage (Prospecting, Engaging, Won, Lost)
  - engage_date
  - close_date
  - close_value
  - Relacionamentos:
  - accounts ← sales_pipeline → products
  - sales_pipeline → sales_teams

Informações complementares:
- accounts.csv: setor, receita, número de funcionários, localização, empresa-mãe
- products.csv: catálogo com série e preço
- sales_teams.csv: vendedores, managers e escritórios regionais

Seu tarefa:
Projete e implemente uma solução funcional completa que seja útil para vendedores reais e ajude a vendedor

A solução pode ser, por exemplo:
- aplicação web
- dashboard interativo
- API + interface simples
- CLI com ranking de prioridades
- ou
- planilha inteligente
- ou outro formato, desde que funcione de verdade.

Requisitos obrigatórios:
1. Precisa rodar de verdade.
2. Precisa usar os dados reais do dataset.
3. Precisa ter lógica de scoring/priorização além de simplesmente ordenar por valor
4. Precisa explicar por que cada deal tem score alto ou baixo
5. Precisa incluir documentação mínima:
   - setup
   - dependências
   - comandos
   - como executar
   - lógica de scoring
   - limitações
6. Precisa incluir process log mostrando como IA foi usada na construção

Quero que você entregue a resposta organizada nas seções abaixo:

1. Proposta de solução
   Defina qual tecnologia será usada e por que ela é adequada para vendedores
   Priorize simplicidade, velocidade de implementação e utilidade prática.

2. Arquitetura da solução
   Descreva:
   - componentes
   - fluxo de dados
   - leitura e tratamento dos CSV
   - regras de negócio
   - interface para o usuário final

3. Estratégia de scoring/priorização
   Crie uma lógica clara e defensável para ranquear oportunidades, combinando fatores como:
   - estágio do deal
   - tempo parado no estágio
   - histórico implícito do fechamento por vendedor/região/produto/setor
   - valor potencial
   - probabilidade de vitória
   - peso de risco e atraso de operacional
   A lógica deve:
   - funcionar com os dados disponíveis
   - ser interpretável
   - gerar justificativas legíveis por humanos

4. Funcionalidades mínimas da ferramenta
   Inclua pelo menos:
   - visão do pipeline
   - ranking de oportunidades prioritárias
   - explicação do score por oportunidade
   - filtros por vendedor, manager e escritório regional
   - destaque para oportunidades "quentes", "em risco" e "esfriando"

5. Código funcional
   Forneça o código completo da solução escolhida, com estrutura de arquivos organizada.
   O código deve ser executável e usar os CSVs reais como entrada.

6. Setup e execução
   Explique passo a passo:
   - dependências
   - instalação
   - comandos
   - como abrir e usar a solução

7. Documentação da lógica
   Explique:
   - quais critérios foram usados no score
   - por que esses critérios fazem sentido para vendas
   - como o vendedor deve interpretar a priorização

8. Limitações
   Descreva:
   - limitações técnicas
   - limitações dos dados
   - o que falta para produção em escala

9. Process log
   Monte um process log exemplar, em formato de narrativa estruturada, contendo:
   - ferramenta(s) de AI utilizada(s)
   - prompts usados em cada etapa
   - como o problema foi quebrado
   - erros e/ou bolhas
   - ajustes realizados
   - iterações
   - decisões humanas finais

10. Entrega final
   Organize a resposta final como um pacote de submissão de challenge, contendo:
   - `solution/`
   - `data/`
   - `code/`
   - `documentation/`
   - `process.log`

Versão pronta para copiar e colar
```

Sugestões para fortalecer ainda mais essa prompt

Eu incluiria estes dois elementos para aumentar muito a qualidade da entrega:

- **forçar uma stack específica**, caso você queira evitar ambiguidade
  Ex: "Use Streamlit + pandas + plotly" ou "Use Reex e Flask/FastAPI no back"
- **pedir saída com estrutura de arquivos completa**
  Ex: "Retorne a árvore de arquivos, o conteúdo de `requirements.txt`, `run.sh`, `README.md`, `app.py`, `scoring.py`, `data_loader.py`"
- **exigir decisão de MVP explícita**
  Isso evita uma resposta superficial tentando ir em "achismo"
- **pedir UX específica ao vendedor**
  Ex: "a primeira tela deve responder: quais 10 deals atacar hoje e por quê?"
- **pedir indicadores de saúde do pipeline**
  Ex: deals estagnados, aging por stage, taxa de conversão por vendedor/região/produto

Minha recomendação de versão mais forte

Para maximizar a chance de uma resposta realmente boa, eu usaria uma versão ainda mais diretiva, dizendo:

- stack preferida
- formato do parser
- critérios mínimos de score
- layout mínimo da interface
- exemplo de explicação por deal
- entrega em arquivos

Posso fazer isso nas próximas iterações.

Quando quiser, digo "**próximo**" e eu avanço para o **Prompt 2**, no framework **ROSE**.

Chatgpt pode cometer erros. Considere verificar informações importantes.